



Prim's Algorithm

Minimum Cost Spanning Tree: Prim's Algorithm

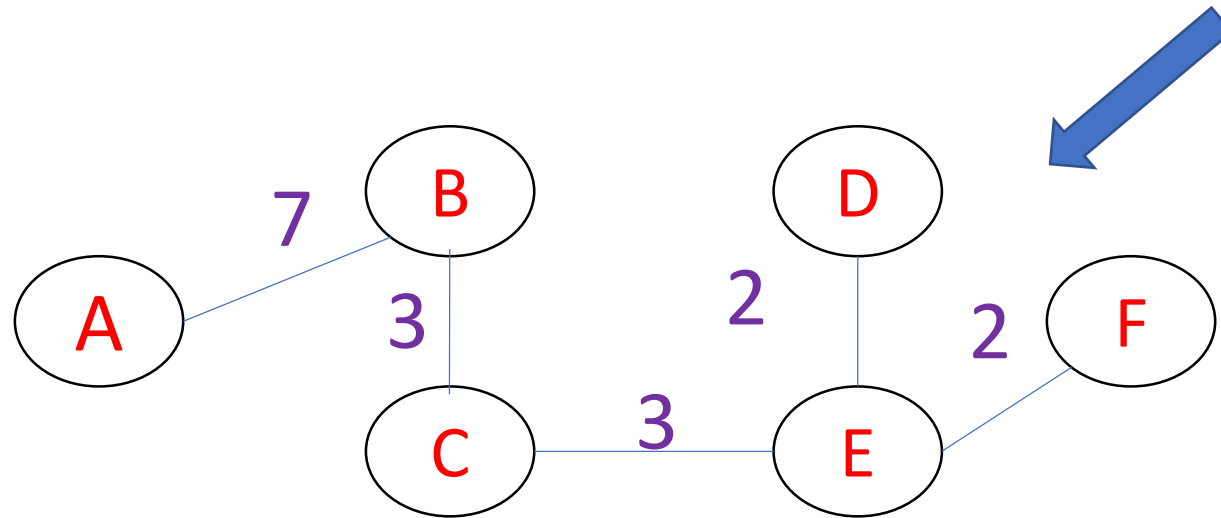
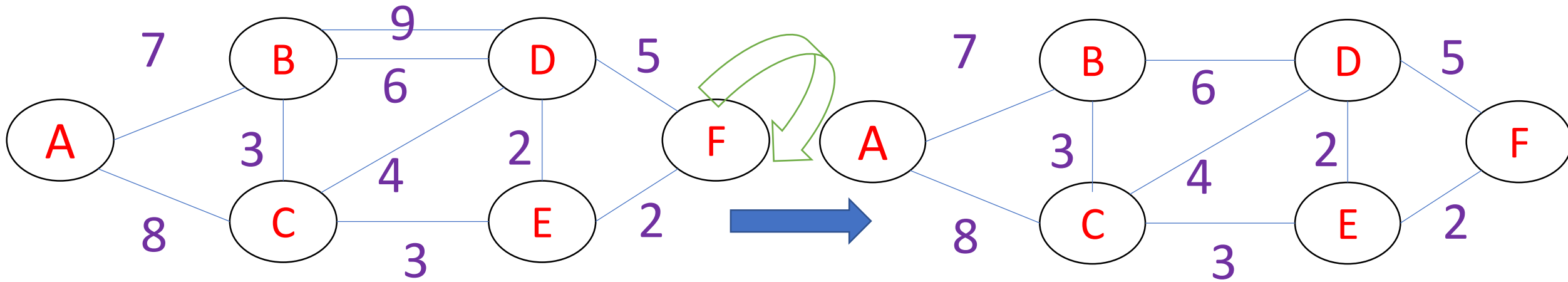
1. Remove loops and parallel edges [Keep min weight]
2. Maintain two disjoint sets of vertices. One containing vertices that are in the growing spanning tree and other that are not in the growing spanning tree.
3. While adding new edge , select edge with minimum weight out of edges from already visited vertices (no cycle allowed).
4. Stop at $[n-1]$ edges.

Note: Step 3 can be Implemented using priority queue.

Timing Analysis of Prim's Algorithm

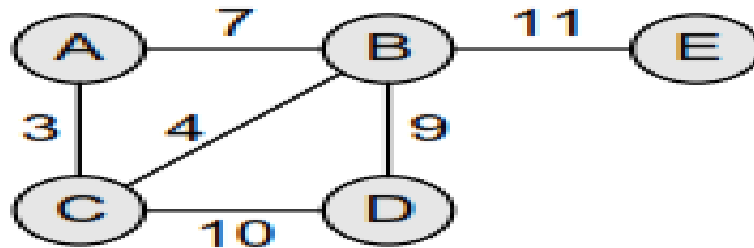
The time complexity of the Prim's Algorithm is $O((V+E) \cdot \log V)$ because each vertex is inserted in the priority queue only once and insertion in priority queue take logarithmic time.

Minimum Cost Spanning Tree-Prim's Algorithm

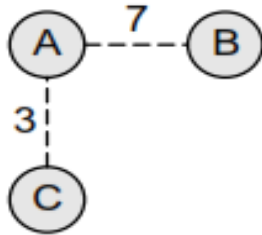


1. Remove loops and parallel edges [Keep min weight]
2. While adding new edge, select edge with minimum weight out of edges from already visited vertices (no cycle allowed).
3. Stop at $[n-1]$ edges.

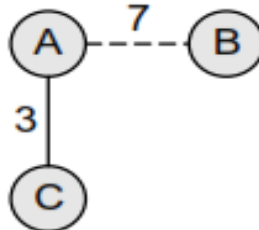
MST using Prim's algorithm after each stage



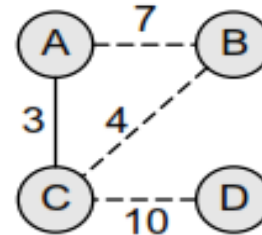
Step 1



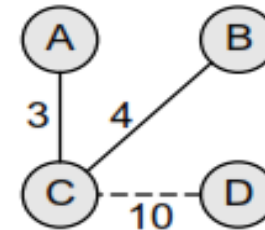
Step 2



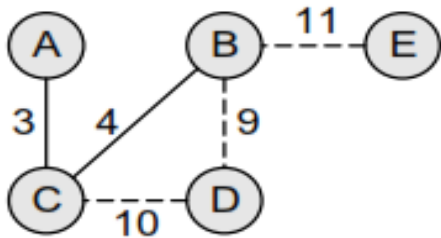
Step 3



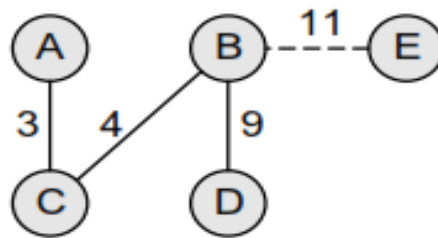
Step 4



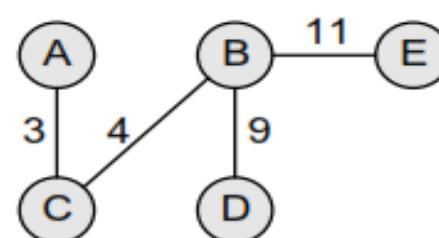
Step 5



Step 6

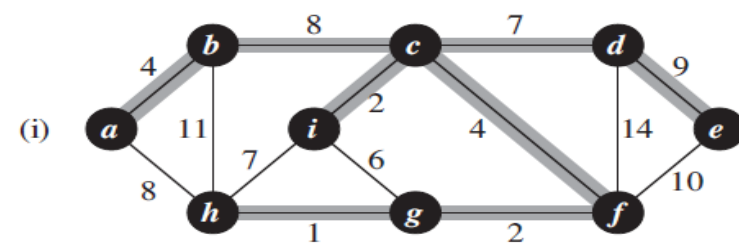
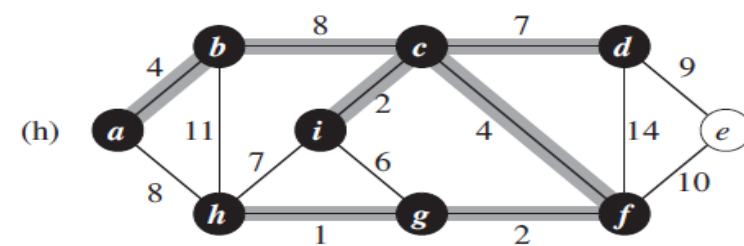
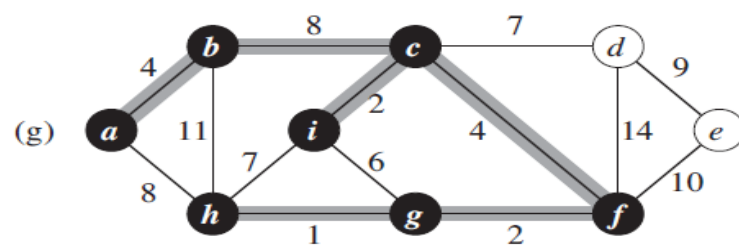
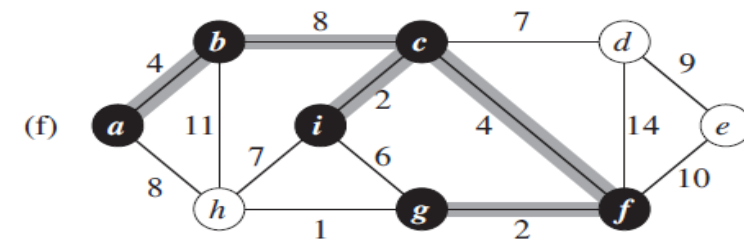
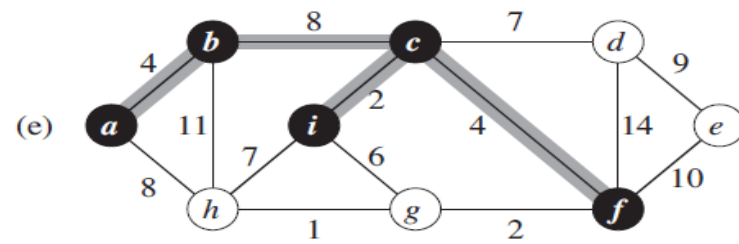
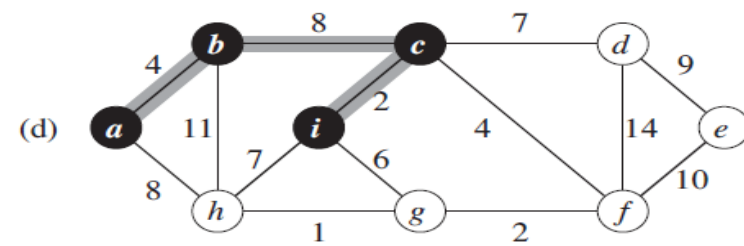
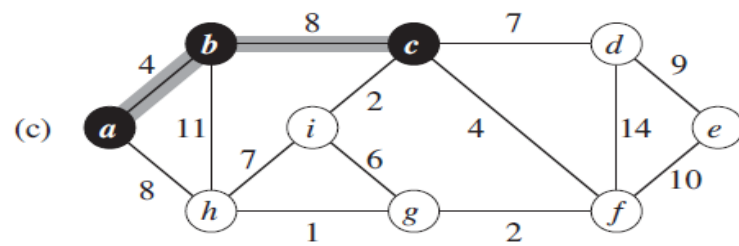
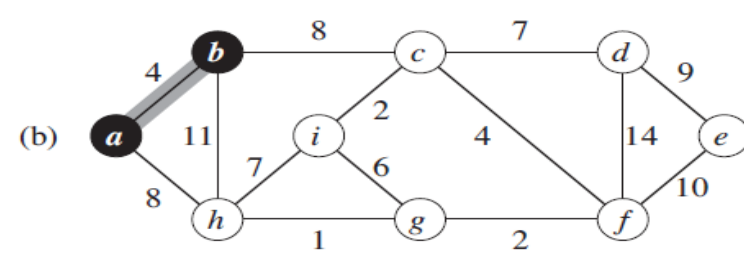
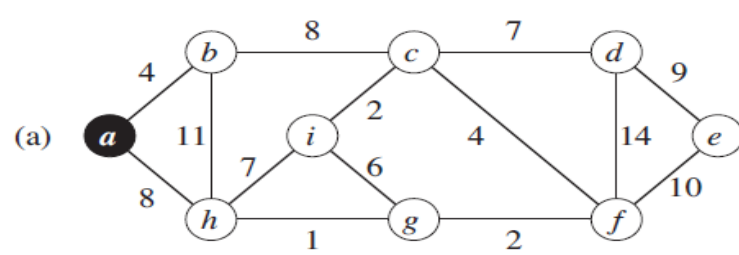


Step 7



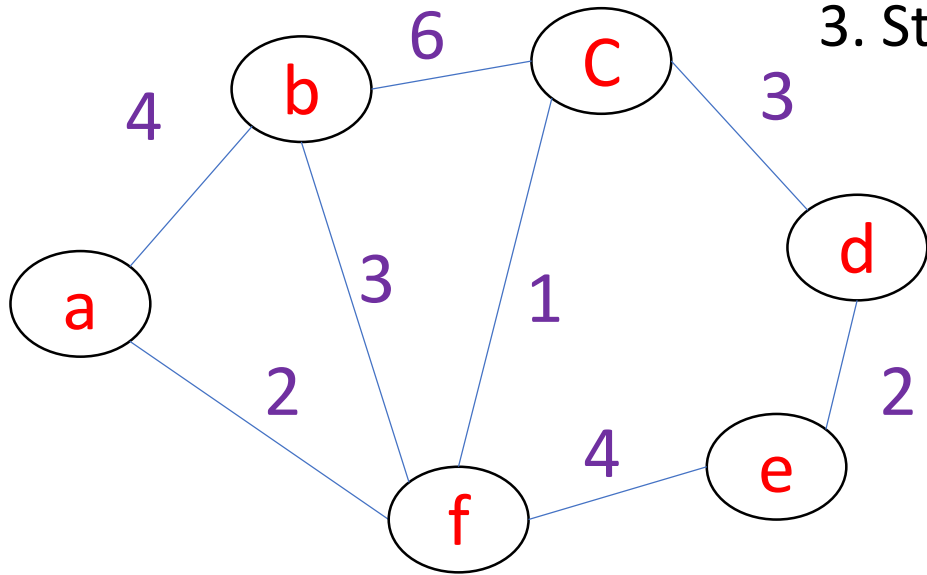
Step 8

Cost=3+4+9+11=27



Minimum Spanning Trees Algorithm- Prim's

1. Remove loops and parallel edges [Keep min weight]
2. While adding new edge, select edge with minimum weight out of edges from already visited vertices (no cycle allowed).
3. Stop at $[n-1]$ edges.



Minimum Spanning Trees Algorithm- Prim's

1. Remove loops and parallel edges [Keep min weight]
2. While adding new edge, select edge with minimum weight out of edges from already visited vertices (no cycle allowed).
3. Stop at $[n-1]$ edges.

